COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/11/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **SORTING ALGORITHMS**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
const int maxSize = 100;

// Function prototype
void displayArr(const int arr[]);

int main() {
    //declare array
    int originalArr[maxSize];
    int quickArr[maxSize];
    int mergeArr[maxSize];
    int shellArr[maxSize];

    //random number generation
    srand(time(0));

    //fill array with random values
    for (int i = 0; i < maxSize; i++) {
        originalArr[i] = rand() % 1000;

        //copy array values
        quickArr[i] = originalArr[i];
        mergeArr[i] = originalArr[i];
        shellArr[i] = originalArr[i];
    }

    //Display unsorted array
    std::cout << "----------------UNSORTED ARRAY----------------\n";

    displayArr(originalArr);
```

```
----------------UNSORTED ARRAY----------------
289     404     424     62      528     352     200     20      408     136
74      54      190     407     611     240     656     696     976     457
733     255     644     47      121     869     55      777     668     378
912     0       988     305     644     232     679     214     624     507
637     18      771     976     685     51      695     874     216     901
778     3       235     622     872     323     993     409     955     28
296     729     797     400     210     800     197     44      375     952
561     558     169     786     595     166     432     6       423     153
343     890     676     76      727     942     553     287     354     519
257     663     923     626     250     384     812     124     883     595
```

Observations	This is basically just declaring array size, and copying its contents to ensure similar samples are used for all sorting algorithms. The arrays are filled with random values using srand function.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
//Shell Sort
template<typename T>
void shellSort(T array[], int size) {
    for (int gap = size / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < size; i++) {
            T temp = array[i];
            int key = i;
            while (key >= gap && array[key - gap] > temp) {
                array[key] = array[key - gap];
                key -= gap;
            }
            array[key] = temp;
        }
    }
}
```

```
std::cout << "\n Shell Sort Result \n";
displayArr(shellArr);
```

```
 Shell Sort Result
0       3       6       18      20      28      44      47      51      54
55      62      74      76      121     124     136     153     166     169
190     197     200     210     214     216     232     235     240     250
255     257     287     289     296     305     323     343     352     354
375     378     384     400     404     407     408     409     423     424
432     457     507     519     528     553     558     561     595     595
611     622     624     626     637     644     644     656     663     668
676     679     685     695     696     727     729     733     771     777
778     786     797     800     812     869     872     874     883     890
901     912     923     942     952     955     976     976     988     993
```

Observations	Shell sort is like an insertion short but it uses a gap which could be inconsistent at times. Basically, it compares the two values separated by the gap, and swaps them if j is lower than i. The best time complexity for shell sort is O(n log n) and the worst time case is $O(N^2)$.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
//Merge helper function
template<typename T>
void merge(T array[], int left, int mid, int right) {
    int sizeL = mid - left + 1;
    int sizeR = right - mid;

    T arrL[MAX_STACK_SIZE];
    T arrR[MAX_STACK_SIZE];

    for (int i = 0; i < sizeL; i++) {
        arrL[i] = array[left + i];
    }
    for (int j = 0; j < sizeR; j++) {
        arrR[j] = array[mid + 1 + j];
    }

    int i = 0, j = 0, k = left;

    while (i < sizeL && j < sizeR) {
        if (arrL[i] <= arrR[j]) {
            array[k] = arrL[i];
            i++;
        } else {
            array[k] = arrR[j];
            j++;
        }
        k++;
    }

    while (i < sizeL) {
        array[k] = arrL[i];
        i++;
        k++;
    }

    while (j < sizeR) {
        array[k] = arrR[j];
        j++;
        k++;
    }
}

//Merge Sort
template<typename T>
void mergeSort(T array[], int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(array, left, mid);
    mergeSort(array, mid + 1, right);
    merge(array, left, mid, right);
}
```

	``
	std::cout << "\n Merge Sort Result \n";
	displayArr(mergeArr);
	``
	``
	Merge Sort Result
	0 3 6 18 20 28 44 47 51 54
	55 62 74 76 121 124 136 153 166 169
	190 197 200 210 214 216 232 235 240 250
	255 257 287 289 296 305 323 343 352 354
	375 378 384 400 404 407 408 409 423 424
	432 457 507 519 528 553 558 561 595 595
	611 622 624 626 637 644 644 656 663 668
	676 679 685 695 696 727 729 733 771 777
	778 786 797 800 812 869 872 874 883 890
	901 912 923 942 952 955 976 976 988 993
	``

Observations	Merge sort does a divide and conquer approach. It divides the main array into sub-array, until it can only consist of 2 values. Then it compares the 2 values, compares it again to other sub-groups until it returns all the values of the array. The best and worst time complexity are the same at O(n log n).

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```cpp
//Partition helper
template<typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            //swapping
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    //swapping for pivot
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

//Quick Sort
template<typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```cpp
std::cout << "\n Quick Sort Result \n";
displayArr(quickArr);
```

```
 Quick Sort Result
0       3       6       18      20      28      44      47      51      54
55      62      74      76      121     124     136     153     166     169
190     197     200     210     214     216     232     235     240     250
255     257     287     289     296     305     323     343     352     354
375     378     384     400     404     407     408     409     423     424
432     457     507     519     528     553     558     561     595     595
611     622     624     626     637     644     644     656     663     668
676     679     685     695     696     727     729     733     771     777
778     786     797     800     812     869     872     874     883     890
901     912     923     942     952     955     976     976     988     993
```

Observations	Uses a partition helper which dictates that the value higher than the pivot goes to the right, and the value less than the pivot, goes to the left. The pivot will end up in its correct position after the 1st iteration. The best time complexity is O(n log n). The worst time complexity is $O(N^2)$.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:

```
//Partition then Merge Sort
template <typename T>
void partiMerge(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        mergeSort(arr, low, pi - 1);
        mergeSort(arr, pi + 1, high);
        merge(arr, low, pi, high);
    }
}

//Partition then Shell Sort AA
template<typename T>
void partiShell(T arr[], int size) {
    int pi = partition(arr, 0, size - 1);
    shellSort(arr, size);
}
```

Figure 1. Code snippet for proof of Problem 1

Analysis:

Based on this Code Snippet, once a partition using a pivot is done by the quick sort, other sorting algorithms can be used to sort the left and right partition of the array.

Problem 2:

```
----------PROBLEM 2: RUNTIME & TIME COMPLEXITY----------
Input Array (N = 15):
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
[EMPIRICAL RUNTIME MEASUREMENTS]
Quick Sort Execution Time : 500 ns
Merge Sort Execution Time : 1200 ns
Shell Sort Execution Time : 400 ns
```

Figure 2. Output for proof of Problem 2

Analysis:

Based on the output, the fastest runtime is the shell sort. This may be due to the small number of indices that the array has. All three sorting algorithm has O(n log n) time complexity but differs in runtime maybe because of other circumstances such as the array size.

Conclusion:

Overall, I learned a lot about additional sorting algorithms such as quick sort, merge sort, and shell sort. These sorting algorithms all have their own advantages depending on the array size. However, I think that merge sort is the most consistent since it has O(n log n) as both its worst and best time complexity unlike the other two. All in all, I think that these sorting algorithms are very helpful in data structures and algorithm since it really aids us in managing and organizing data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.